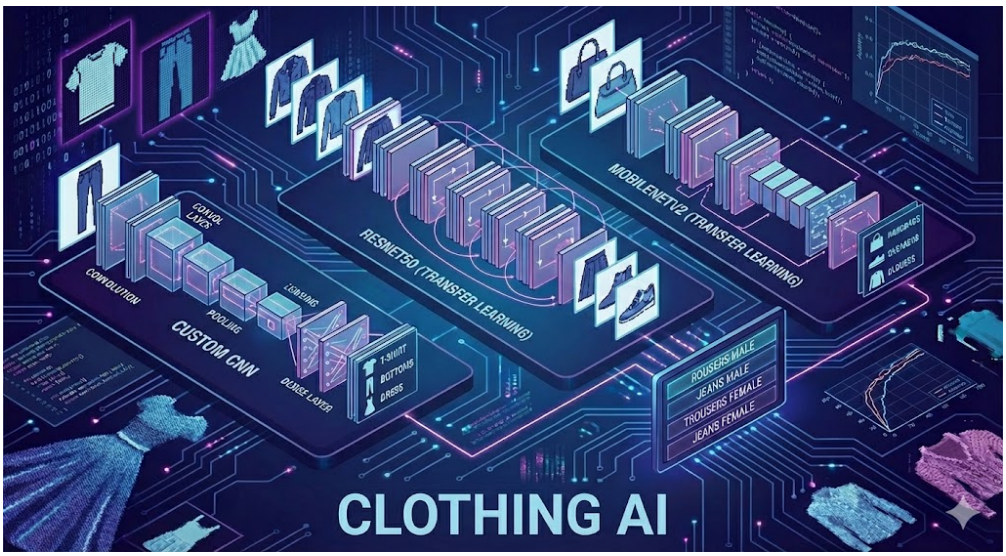
CLOTHING AI

Group Members

Qadeem Khan
Syed Sameed Ahmed
Arham Sajjad
Sajeed Hussain



Submitted To

Professor Joaqin Jorge Rodriguez

Computer Vision and Robotics
Université Bourgogne Europe

December 12, 2025

Contents

1 Introduction **3**
1.1 Objective of the Study . . . 3
1.2 Problem Statement . . . 3

2 Literature Review **3**
2.1 Traditional Approaches to Image Classification . . . 4
2.2 Convolutional Neural Networks (CNNs) . . . 4
2.3 Transfer Learning and Pretrained Models . . . 4

3 Dataset Used **4**

4 Methodology **6**
4.1 End-to-End data flow . . . 6
4.2 Overview of the Approach . . . 6
4.3 Custom CNN Model Architecture . . . 6

5 Result Discussion **11**
5.1 Custom CNN Model Result Discussion . . . 11
5.2 MobileNetV2 Model Result Discussion . . . 12
5.3 ResNet50 Model Result Discussion . . . 14
5.4 Model Comparison . . . 14
5.5 Conclusion . . . 16

List of Figures

1 Sample of images used in the training . . . 5
2 Diagram of the Custom CNN Architecture showing the sequential flow of convolutional, pooling, dense, and output layers . . . 7
3 MobileNetV2 Architecture [11] . . . 9
4 ResNet50 Architecture [6] . . . 10
5 Custom CNN Accuracy & Loss . . . 12
6 Custom CNN Confusion Matrix . . . 13
7 MobileNetV2 Accuracy & Loss . . . 13
8 MobileNetV2 Confusion Matrix . . . 14
9 ResNet50 Accuracy & Loss . . . 15
10 ResNet50 Confusion Matrix . . . 15
11 Model comparison plot . . . 16

1 Introduction

Deep learning has emerged as one of the most effective approaches for solving image classification problems due to its ability to automatically learn hierarchical feature representations from raw pixel data. Convolutional Neural Networks (CNNs), in particular, have demonstrated state-of-the-art performance across various domains, including medical imaging, object recognition, retail inventory systems, and real-time visual applications [8, 7]. Despite the availability of many modern pretrained architectures, designing an efficient model for a specific dataset still requires careful experimentation with architectural depth, convolutional filter design, activation functions, regularization strategies, and optimization techniques.

This project focuses on developing and evaluating several CNN-based architectures for a three-class fashion image classification task. The evaluation includes both a custom-designed CNN and two state-of-the-art pretrained models: ResNet50, and MobileNetV2. The motivation behind comparing these models lies in understanding how architectural choices influence training behavior, generalization performance, computational efficiency, and robustness. By benchmarking a custom architecture against established transfer-learning models, this study highlights the trade-offs between model size, expressive capacity, accuracy, and training time.

1.1 Objective of the Study

The primary objective of this research is to systematically investigate how architectural depth, convolutional design choices, and hyperparameter tuning affect image classification performance on the selected dataset. Specifically, the study aims to:

1. Construct and train multiple custom CNN architectures with varying depth and complexity.
2. Fine-tune and evaluate pretrained models (ResNet50, and MobileNetV2) for comparison.
3. Assess all models using standard evaluation metrics including accuracy and loss.
4. Analyze the impact of model capacity on underfitting, overfitting, and generalization.
5. Identify the best-performing model for the dataset and establish baseline results for future work.

The scope of this project includes model construction, training, evaluation, and comparative analysis using TensorBoard and statistical performance metrics. The insights gained contribute to understanding effective CNN design, the practical benefits of transfer learning for small-to-medium datasets, and considerations for real-world model deployment.

1.2 Problem Statement

Distinguishing between male and female trousers or jeans is challenging due to subtle design similarities, variations in styling, and overlapping visual features. Human classification can be inconsistent, especially when handling large collections or visually ambiguous items. For fashion retailers and e-commerce platforms, misclassification affects product organization, searchability, and automated tagging workflows. Therefore, there is a need for an accurate, scalable image-based gender classification system. This research evaluates and compares custom CNNs with state-of-the-art pretrained architectures to determine which model best supports reliable automated clothing categorization.

2 Literature Review

Image classification has long been a central task in computer vision, with approaches evolving from traditional machine-learning pipelines to modern deep-learning systems. This section reviews key developments, including traditional feature-engineering methods, the foundations of Convolutional Neural Networks (CNNs), and relevant state-of-the-art architectures.

2.1 Traditional Approaches to Image Classification

Before the advent of deep learning, image classification primarily relied on manually engineered feature extraction techniques such as Scale-Invariant Feature Transform (SIFT) [9], Histogram of Oriented Gradients (HOG) [4], Local Binary Patterns (LBP) [10], and Bag-of-Visual-Words (BoVW) [12]. These handcrafted features were typically paired with classical machine-learning algorithms including Support Vector Machines (SVM) [2], Random Forests [1], or k-Nearest Neighbors (kNN) [3].

Although these approaches performed well for certain tasks, they suffered from significant limitations. They required domain expertise to design suitable features and struggled with complex visual patterns such as texture variation, illumination changes, and nonlinear relationships. More importantly, they lacked the ability to learn hierarchical feature representations, which limited their scalability to large, diverse datasets. These limitations paved the way for deep learning and CNN-based architectures [7].

2.2 Convolutional Neural Networks (CNNs)

CNNs revolutionized image-processing tasks by introducing learnable convolutional filters that capture local spatial patterns such as edges, textures, and shapes [8]. Through weight sharing, the same filter is applied across the entire image, reducing the number of parameters and improving generalization. Early CNN layers detect simple features, while deeper layers combine them to form more abstract semantic representations [5].

As networks become deeper, their ability to learn complex visual patterns increases. However, deeper architectures also face challenges such as vanishing gradients, training instability, and increased risk of overfitting. Modern architectures introduce innovative solutions to these challenges. For example, ResNet employs residual skip connections [6], MobileNetV2 uses depthwise separable convolutions and inverted residuals [11], and EfficientNet applies compound scaling of network depth, width, and resolution [13]. These innovations enable deeper networks to train effectively while maintaining computational efficiency.

2.3 Transfer Learning and Pretrained Models

Transfer learning has become a widely adopted strategy for image classification, especially for datasets that are small or domain-specific. By leveraging models pretrained on large-scale datasets such as ImageNet, networks can extract rich and generalizable visual features from new inputs [14]. Fine-tuning these pretrained networks not only boosts performance but also significantly reduces training time.

ResNet50 [6] introduces residual learning to address the degradation problem in deep networks. MobileNetV2 [11] is optimized for low-resource environments through efficient convolutional operations.

3 Dataset Used

The dataset used in this study is an open-source dataset obtained from Roboflow https://universe.roboflow.com/yoloproject-o8dlo/fashionclassification-zvdad, and consists of $11,733$ colored images of trousers and jeans, each with a resolution of 224×224 pixels and captured against a consistent plain background. The objective is to classify each garment by gender and type, independent of the individual wearing it. The dataset includes four distinct classes: *Trousers Male*, *Jeans Male*, *Trousers Female*, and *Jeans Female*. The dataset was divided into a training set of 10,263 images, a validation set of 738 images, and a test set of 732 images. Table 1 summarizes the class distribution across the dataset splits.

Table 1: Class distribution across dataset splits

Class	**Label**	**Training**	**Validation**	**Test**
Trousers Male	0	2,529	195	202
Jeans Male	1	2,520	193	191
Trousers Female	2	2,586	179	162
Jeans Female	3	2,628	171	177

As shown in the table, the dataset is relatively balanced, with only minor variations across classes, particularly in the validation and test sets. These slight imbalances were accounted for during training

through the careful selection of loss functions and, when necessary, class weighting and augmentation. The proportional split of approximately 70% training, 15% validation, and 15% testing ensures sufficient data for training high-capacity models while providing reliable monitoring for early stopping and hyperparameter tuning without compromising the test set. Figure 1 presents random samples from the dataset.

Figure 1: Sample of images used in the training

To enhance model performance, the dataset was further expanded and preprocessed using a series of transformations, resulting in a total of 11,733 images organized in a folder-based annotation format. Each image underwent the following preprocessing steps:

- Auto-orientation using EXIF metadata (with orientation stripping)
- Resizing to 224×224 pixels using stretch scaling
- Contrast enhancement using adaptive histogram equalization

To improve robustness and increase dataset diversity, data augmentation techniques were applied to generate three additional versions of each image. Each augmented image included:

- 50% probability of horizontal flipping
- Random rotation between $-15°$ and $+15°$
- Random brightness variation between -15% and $+15\%$
- Random Gaussian blur between 0 and 2.5 pixels

These augmentations help the model generalize effectively by simulating real-world variations in orientation, illumination, and minor distortions while preserving key garment characteristics relevant for classification.

4 Methodology

This chapter describes the methodology adopted for developing, training, and evaluating the classification models used in this research. It presents the custom-built Convolutional Neural Network (CNN), the selected pretrained architectures, the hyperparameter tuning process, and the evaluation procedures. The goal is to provide a transparent and reproducible overview of the modeling pipeline.

4.1 End-to-End data flow

The project implements a modular image classification system for clothing recognition using a React frontend, Flask backend, and TensorFlow-based CNN models. The frontend serves as the primary user interface, providing users with an intuitive platform to upload images of clothing. Users interact with a responsive upload component, which previews the selected image and triggers an HTTP POST request to the backend upon submission. This interface ensures a smooth and user-friendly experience while maintaining real-time feedback for uploading and classification progress.

The backend, built with Flask, functions as the core application server, managing image processing, model inference, and communication with the frontend. Upon receiving an image URL from the frontend, the backend validates the input and downloads the image. The image is preprocessed by resizing, normalizing pixel values, and converting it into a format suitable for the TensorFlow CNN models. The preprocessed image is then fed into a trained model to predict the clothing category, and the backend returns a structured JSON response with the predicted class and confidence score. Additionally, uploaded images are scheduled for deletion from the cloud storage after classification to maintain storage efficiency and privacy.

4.2 Overview of the Approach

The methodology consists of two major components:

1. Custom CNN Model Development

- A lightweight CNN was designed from scratch.
- Hyperparameter tuning was performed using TensorBoard HParams.
- The best-performing architecture (Run–15) was selected.

2. Transfer Learning with Pretrained Models

Two state-of-the-art pretrained models were selected for fine-tuning:

- MobileNetV2
- ResNet50V2

Both approaches were evaluated using identical training, validation, and test splits to ensure a fair comparison.

4.3 Custom CNN Model Architecture

The custom CNN was designed to serve as a baseline architecture for comparison against state-of-the-art pretrained models. The optimized version of the network, identified as Run–15, consists of two convolutional layers with 64 filters and 5×3 kernels, each followed by a 2×2 max-pooling layer. These convolutional layers extract low-level features such as edges and textures, as well as higher-level patterns necessary for distinguishing between trousers and jeans and between male and female categories.

The resulting feature maps are flattened and passed through a fully connected dense layer with 256 units and ReLU activation. The network concludes with a four-unit output layer producing logits for classification. Figure 4.1 illustrates the architecture.

The architecture was designed through systematic hyperparameter tuning using TensorBoard HParams. The search evaluated:

- Filter sizes: 3×3 and 5×5
- Number of filters: 32 and 64
- Dense layer sizes: 256 and 512

Across 16 experimental runs, Run–15 (64 filters, 5×5 kernel, 256 dense units) achieved the best balance between accuracy and overfitting. Larger kernels captured broader texture patterns typical of denim and trousers, while avoiding unnecessary parameter inflation.

The model was trained using Sparse Categorical Crossentropy, which is appropriate for multi-class integer-labeled tasks. ReLU activations were used in all convolutional and dense layers to enhance non-linear feature learning. The output layer produced raw logits, which the loss function internally converted into probabilities.

Model performance was monitored with accuracy and loss curves, indicating stable convergence without oscillation. Confusion matrices were logged via TensorBoard to track class-level performance, and early stopping was applied to prevent overfitting and preserve the best-performing weights. This custom CNN serves as a strong baseline against which the pretrained architectures are evaluated.

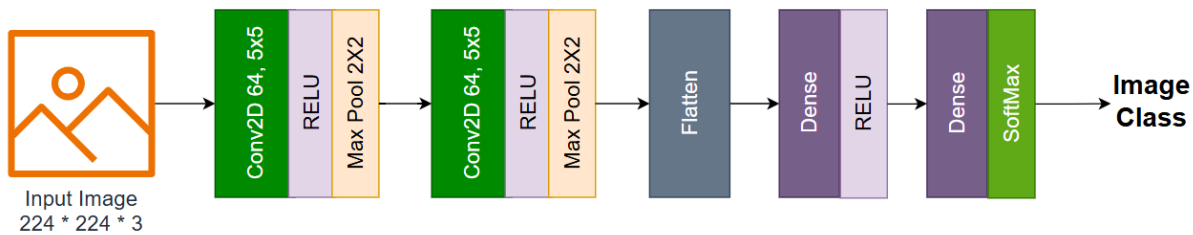


Figure 2: Diagram of the Custom CNN Architecture showing the sequential flow of convolutional, pooling, dense, and output layers

Formally, the output of a convolutional layer (as illustrated in Figure 2) can be computed as:

$$y_k(i,j) = \sigma\left(\sum_{c=1}^{C}\sum_{m=0}^{F-1}\sum_{n=0}^{F-1} W_{k,c}(m,n)\, x_c(i+m, j+n) + b_k\right) \tag{1}$$

Where:

- $y_k(i,j)$ is the output at spatial position (i,j) of the k-th feature map.
- σ is the activation function (e.g., ReLU).
- F is the filter (kernel) size.
- C is the number of input channels.
- $W_{k,c}(m,n)$ is the weight at position (m,n) in the filter connected to channel c for feature map k.
- $x_c(i+m, j+n)$ is the input value at position $(i+m, j+n)$ in channel c.
- b_k is the bias term for feature map k.

Pooling layers reduce the spatial dimensions of feature maps, lowering computational cost and model parameters. Max pooling is the most commonly used pooling method. Its output is computed as:

$$p_k(i,j) = \max_{0 \le m,n < S} x_k(i+m, j+n) \tag{2}$$

Where:

- $p_k(i,j)$ is the pooled output at position (i,j) of feature map k.

- S is the pooling window size (e.g., $S = 2$).
- $x_k(i, j)$ is the input feature map to the pooling layer.

After several convolutional and pooling layers, the output is flattened and passed into fully connected (dense) layers. A fully connected layer performs:

$$z_j = \sigma\left(\sum_{i=1}^{N} w_{ij}\, a_i + b_j\right) \tag{3}$$

Where:

- z_j is the output of neuron j in the current layer.
- σ is the activation function.
- w_{ij} is the weight connecting neuron i (previous layer) to neuron j.
- a_i is the activation of neuron i in the previous layer.
- b_j is the bias for neuron j.

4.4 Pretrained Model Selection and Justification

Three pretrained models were selected based on performance, computational efficiency, and suitability for small-to-medium scale datasets.

4.4.1 MobileNetV2

MobileNetV2 was selected due to its lightweight, computationally efficient architecture specifically designed for deployment on mobile and edge devices. The model relies heavily on inverted residual blocks and depthwise separable convolutions, which drastically reduce the number of parameters and computational complexity while retaining strong feature extraction capability. These characteristics make MobileNetV2 particularly suitable for smaller input sizes and hardware-constrained environments. Within this study, MobileNetV2 serves as an efficient baseline, offering fast training times, low memory usage, and reliable feature representation when compared to the custom CNN and other pretrained architectures. The core innovation of MobileNetV2 lies in its combination of depthwise separable convolutions, inverted residual blocks, and linear bottlenecks. Depthwise separable convolution serves as the fundamental building block and significantly reduces computational cost compared to standard convolution.

For a standard convolution with input feature map size $H \times W \times M$ and N filters of size $K \times K$, the computational cost is:

$$\text{Cost}_{\text{standard}} = H \times W \times M \times N \times K^2 \tag{4}$$

The depthwise separable convolution factorizes this into two operations:
1. **Depthwise convolution**: applies a $K \times K$ spatial filter to each input channel independently.

$$\text{Cost}_{\text{depthwise}} = H \times W \times M \times K^2 \tag{5}$$

2. **Pointwise (1×1) convolution**: linearly combines the depthwise outputs.

$$\text{Cost}_{\text{pointwise}} = H \times W \times M \times N \tag{6}$$

Thus, depthwise separable convolution drastically reduces the total number of floating-point operations while maintaining strong representational capacity.

MobileNetV2 further enhances efficiency through its inverted residual blocks. Each block expands the number of channels using a 1×1 convolution, applies a depthwise convolution to extract spatial features, and then projects the representation back to a lower-dimensional space using a linear 1×1 convolution.

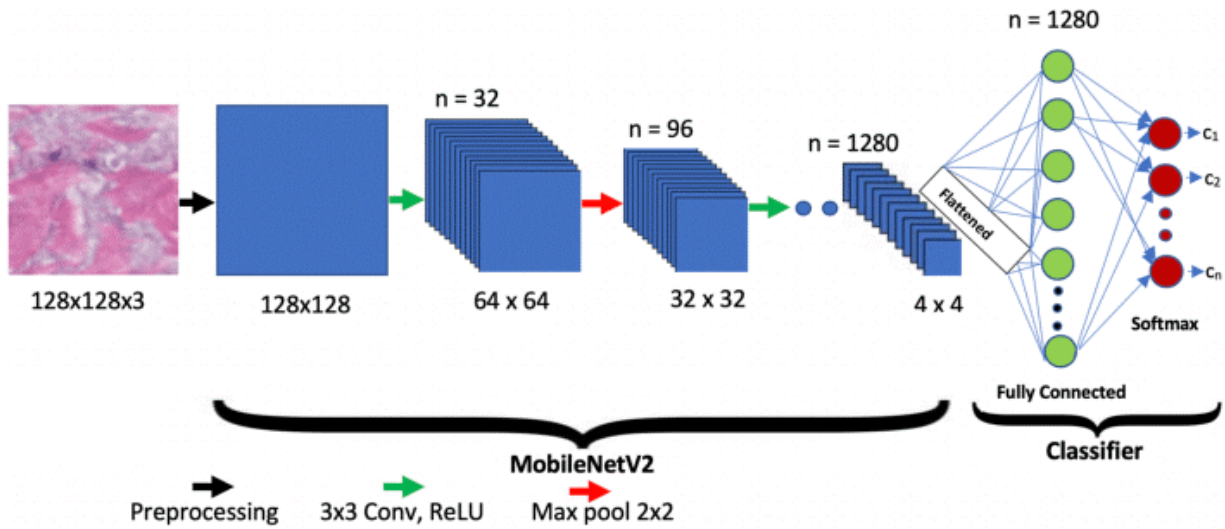


Figure 3: MobileNetV2 Architecture [11]

Let $X \in \mathbb{R}^{H \times W \times C_{\text{in}}}$ be the input and t the expansion factor. The operations in a MobileNetV2 block are:

$$X_1 = \text{ReLU6}\left(\text{Conv}_{1\times1}(X, t \cdot C_{\text{in}})\right) \tag{7}$$

$$X_2 = \text{ReLU6}\left(\text{DWConv}_{3\times3}(X_1)\right) \tag{8}$$

$$X_3 = \text{Conv}_{1\times1}^{\text{linear}}(X_2, C_{\text{out}}) \tag{9}$$

A residual connection is applied whenever $C_{\text{in}} = C_{\text{out}}$ and the stride is 1:

$$Y = X + X_3 \tag{10}$$

This inverted structure (expand $\rightarrow$ depthwise $\rightarrow$ project) reverses the design of traditional residual blocks. By performing nonlinear transformations in the high-dimensional expanded space and compressing information in a low-dimensional bottleneck, the model achieves high efficiency and expressive power.

A key innovation of MobileNetV2 is the use of **linear bottlenecks**. Unlike deeper architectures that apply nonlinear activations such as ReLU after every convolution, MobileNetV2 removes the activation after the projection layer. This prevents information loss in the compressed latent space and ensures that nonlinearities only occur in the expanded high-dimensional representation, preserving essential features.

4.4.3 ResNet50V2

ResNet50 was selected as one of the pretrained baselines due to its deep architecture incorporating skip connections, which effectively address the vanishing gradient problem. These skip connections allow information and gradients to flow more easily through the network, enabling the model to learn deeper and more expressive feature hierarchies.

ResNet50's residual blocks provide robust feature extraction at multiple abstraction levels, making it highly suitable for distinguishing subtle differences across visually similar apparel categories—such as trousers versus jeans—across male and female classes. The model has demonstrated strong performance across numerous computer vision benchmarks, and its stable training behavior makes it an excellent candidate for transfer learning.

After resizing all dataset images to match the required input resolution, ResNet50 was able to fully leverage its hierarchical feature extraction pipeline. Its depth, stability, and representational strength enabled it to serve as a strong comparative architecture in this study.

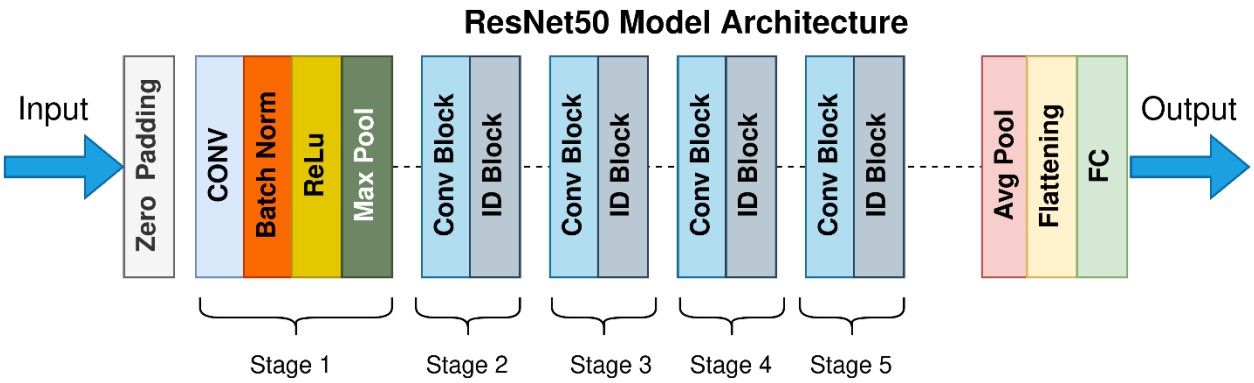


Figure 4: ResNet50 Architecture [6]

The fundamental building block of ResNet50 is the residual block. A residual block is designed to learn a residual mapping $F(X)$ instead of learning the direct underlying mapping $H(X)$. Given an input feature map X, the output of a residual block is defined as:

$$Y = F(X) + X \tag{11}$$

Here, $F(X)$ represents the output of a sequence of convolutional layers, batch normalization, and ReLU activation functions, while the addition of the identity shortcut X ensures that the network can learn perturbations relative to the input. This identity mapping mitigates the degradation problem observed in very deep networks by enabling stable gradient flow during backpropagation.

ResNet50 uses bottleneck residual blocks that reduce computational cost while maintaining strong representational power. Each bottleneck block contains three convolutional layers: a 1×1 convolution to reduce dimensionality, a 3×3 convolution for spatial feature extraction, and a final 1×1 convolution to restore dimensionality. Formally, for an input tensor $X \in \mathbb{R}^{H \times W \times C_{\text{in}}}$, the bottleneck operations are:

$$X_1 = \text{ReLU}\left(\text{BN}\left(\text{Conv}_{1\times1}(X, C_{\text{mid}})\right)\right) \tag{12}$$

$$X_2 = \text{ReLU}\left(\text{BN}\left(\text{Conv}_{3\times3}(X_1, C_{\text{mid}})\right)\right) \tag{13}$$

$$X_3 = \text{BN}\left(\text{Conv}_{1\times1}(X_2, C_{\text{out}})\right) \tag{14}$$

A residual connection is applied as:

$$Y = \text{ReLU}(X + X_3) \tag{15}$$

if $C_{\text{in}} = C_{\text{out}}$; otherwise, a projection shortcut (typically a 1×1 convolution) is used to match dimensions. ResNet50 stacks multiple such bottleneck blocks in four major stages, progressively increasing the number of channels while reducing spatial resolution through stride-2 convolutions. This enables learning of hierarchical features ranging from low-level edges to high-level semantic structures.

4.5 Training Configuration

To ensure a fair and consistent comparison between the custom CNN and all pretrained models, each architecture was trained under identical experimental settings. The primary loss function used was Sparse Categorical Crossentropy with `from_logits=True`. This choice was appropriate because the dataset labels were integer-encoded from 0 to 3, making Sparse Categorical Crossentropy efficient for multi-class classification. Furthermore, this loss integrates well with models that output logits, improving numerical stability by applying the final activation internally.

The Adam optimizer was employed for all models due to its adaptive learning rate mechanism, fast convergence, and robustness when dealing with noisy gradients, particularly at the early stages of training.

A batch size of 32 was used for all experiments, and each model was trained for a maximum of 50 epochs. Early stopping was enabled by monitoring validation loss with a patience of two epochs, ensuring that training halted once improvements plateaued and automatically restoring the best-performing weights. This mechanism also served as an additional safeguard against overfitting.

For all pretrained architectures, transfer learning was applied in a two-stage process. First, the base convolutional layers were loaded with `include_top=False` to remove the ImageNet classifier, and all base layers were frozen to preserve their pretrained weights. A custom classification head was appended, consisting of a GlobalAveragePooling2D layer, a dense layer with 256 ReLU units, and a final dense layer with four output units corresponding to the target classes. Depending on the model, the final layer produced either logits or softmax-activated probabilities. After the classification head had learned meaningful task-specific patterns, the top layers of each pretrained model were selectively unfrozen to enable fine-tuning, allowing high-level features to adapt more effectively to the clothing classification task.

4.6 Evaluation Procedure

All models were evaluated using the same performance metrics and testing pipeline. Validation accuracy was monitored throughout training to assess learning progression, while final test performance was measured using the model evaluate method to obtain both test loss and test accuracy. Confusion matrices were generated after each epoch and logged to TensorBoard to provide detailed insight into class-wise performance and common misclassification patterns.

Both accuracy and loss were computed separately for training, validation, and testing in order to provide a granular understanding of model behavior, especially for assessing inter-class imbalance. Training and validation curves were also recorded to observe convergence trends and verify stability. Each model was logged in its own TensorBoard directory to allow clean visualization and direct comparison across all experiments.

5 Result Discussion

5.1 Custom CNN Model Result Discussion

Our custom CNN models were trained with multiple hyperparameter configurations to identify the best combination. Table 2 summarizes the best training results achieved by each run.

Table 2: Summary of Best Training Results for Custom CNN Models

Run	Filters Conv1	Filters Conv2	Filter Size1	Filter Size2	Dense Units	Train Acc	Train Loss	Val Acc	Val Loss
9	32	32	5	3	256	0.9411	0.1629	0.8103	0.6685
2	32	32	3	3	512	0.8985	0.2655	0.7981	0.6598
5	64	32	3	3	256	0.8738	0.3344	0.7859	0.5779
12	32	32	5	5	512	0.8774	0.3283	0.7886	0.5529
3	32	32	3	5	256	0.8775	0.3301	0.7832	0.6588
6	64	32	3	3	512	0.9032	0.2591	0.7886	0.6007
7	64	32	3	5	256	0.8863	0.3009	0.7764	0.5609
11	32	32	5	5	256	0.8901	0.2946	0.7778	0.5941
10	32	32	5	3	512	0.9256	0.2039	0.7886	0.6145
13	64	32	5	3	256	0.8741	0.3327	0.7846	0.5638
4	32	32	3	5	512	0.9346	0.1820	0.7656	0.6720
14	64	32	5	3	512	0.9034	0.2511	0.7710	0.5788
15	64	64	5	5	256	0.9068	0.2481	0.7696	0.6044
16	64	64	5	5	512	0.9009	0.2679	0.7656	0.6040
1	32	32	3	3	256	0.9227	0.2102	0.7453	0.6565
8	64	64	3	5	512	0.2561	1.3863	0.2317	1.3870

Table 2 presents the best validation accuracy achieved for each run, ranking the runs based on performance.

During the training of the custom CNN model, it was observed that the training accuracy of the best-performing model (Run 9) gradually increased until it peaked at the fourth step, while the training loss decreased steadily, reaching its lowest point at the same step. The validation accuracy and loss remained

Table 3: Best Validation Accuracy for Custom CNN Models

Rank	Run	Filter Size1	Filters	Filter Size2	Dense Units	Best Val Acc	Best Val Loss
1	Run 9	5	32	3	256	0.7954	0.5429
2	Run 2	3	32	3	512	0.7818	0.5756
3	Run 5	3	64	3	256	0.7859	0.5779
4	Run 10	5	32	3	512	0.7886	0.6145
5	Run 11	5	32	5	256	0.7778	0.5941
6	Run 7	3	64	5	256	0.7764	0.5609
7	Run 12	5	32	5	512	0.7886	0.5529
8	Run 3	3	32	5	256	0.7656	0.6008
9	Run 16	5	64	5	512	0.7656	0.6040
10	Run 4	3	32	5	512	0.7656	0.6720
11	Run 13	5	64	3	256	0.7846	0.5638
12	Run 6	3	64	3	512	0.7602	0.6007
13	Run 15	5	64	5	256	0.7696	0.6044
14	Run 1	3	32	3	256	0.7453	0.6565
15	Run 14	5	64	3	512	0.7710	0.5788
16	Run 8	3	64	5	512	0.2317	1.3870

relatively stable across epochs, with only minor fluctuations, indicating consistent model generalization and effective early stopping to prevent overfitting.

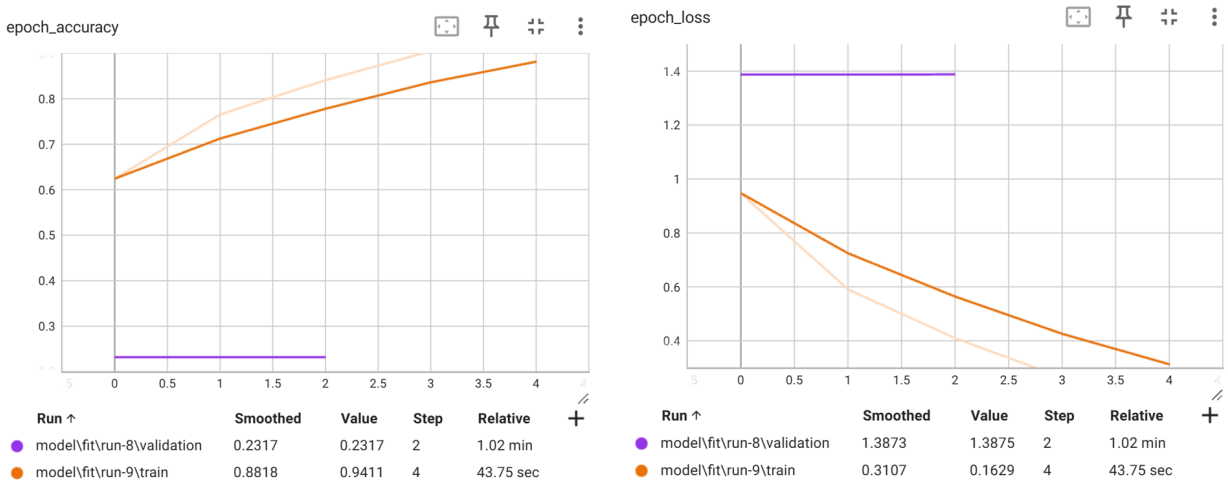


Figure 5: Custom CNN Accuracy & Loss

Observing the confusion matrix in Figure 6, the custom CNN model correctly predicted 75% of Trousers Male, 81% of Jeans Male, 84% of Trousers Female, and 83% of Jeans Female. Misclassifications were relatively low but included cases such as Trousers Male being misclassified as Trousers Female 17 times out of 100, Jeans Male as Jeans Female 12 times out of 100, and Trousers Female as Jeans Female 13 times out of 100.

5.2 MobileNetV2 Model Result Discussion

During the training of the MobileNetV2 model, the training accuracy increased gradually until peaking at the tenth step, while the loss decreased steadily to its lowest point at the same step. Validation accuracy and loss remained stable with minor fluctuations.

The confusion matrix for MobileNetV2 in figure 8 indicates strong performance in classifying male clothing

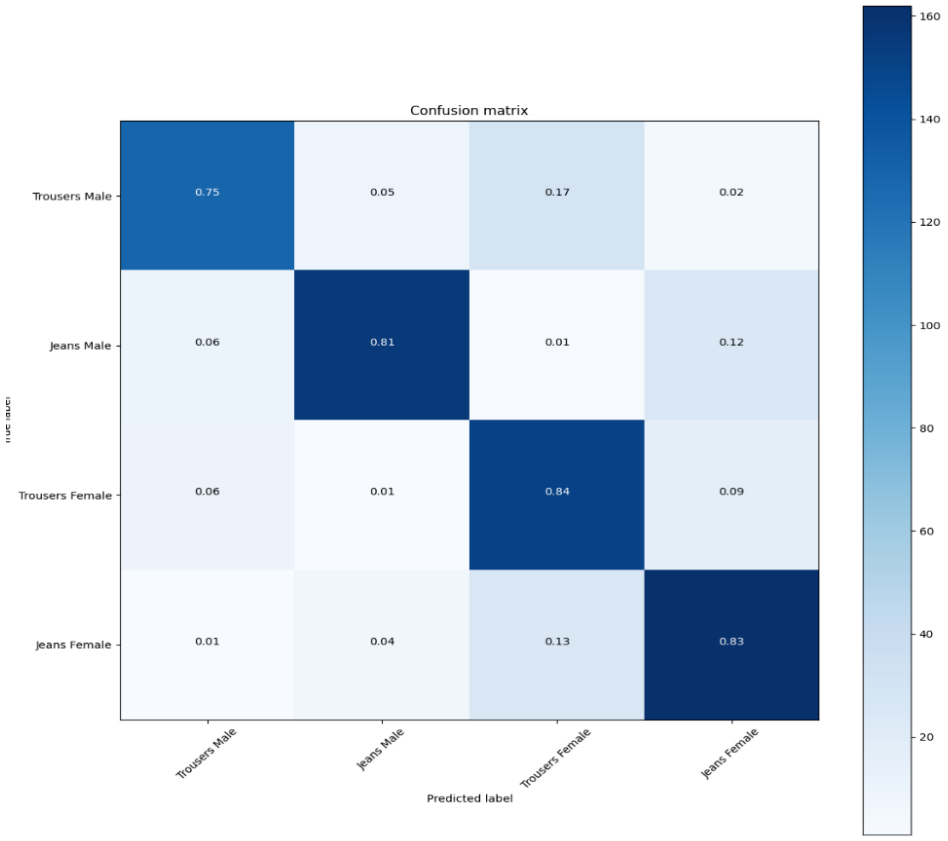


Figure 6: Custom CNN Confusion Matrix

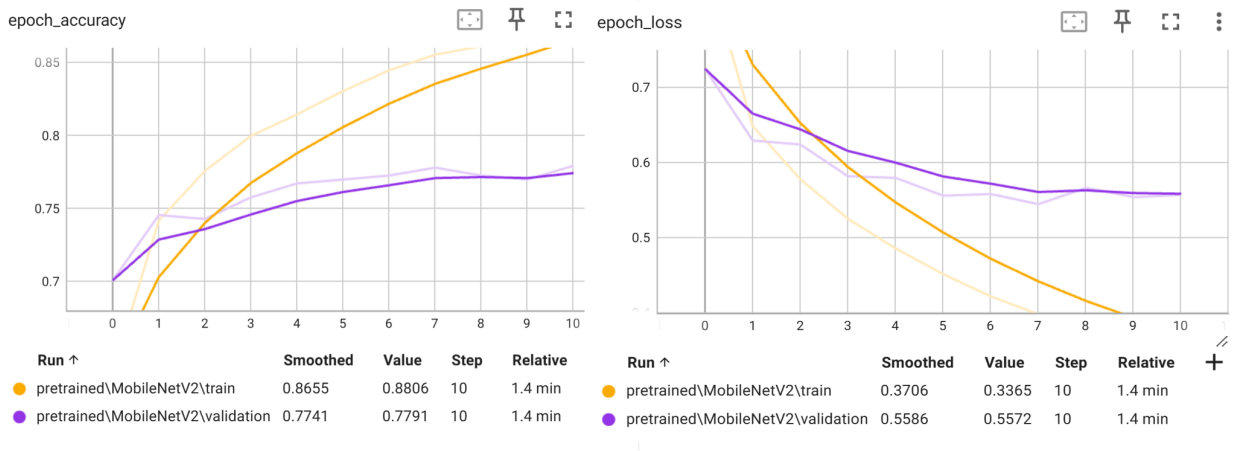


Figure 7: MobileNetV2 Accuracy & Loss

categories, with approximately 85–90% of Trousers Male and Jeans Male correctly identified. Performance on female clothing categories was slightly lower, with 70–75% of Trousers Female and Jeans Female correctly classified. Most errors occurred within the same gender, e.g., Trousers Female misclassified as Jeans Female, accounting for 20–25% of misclassifications.

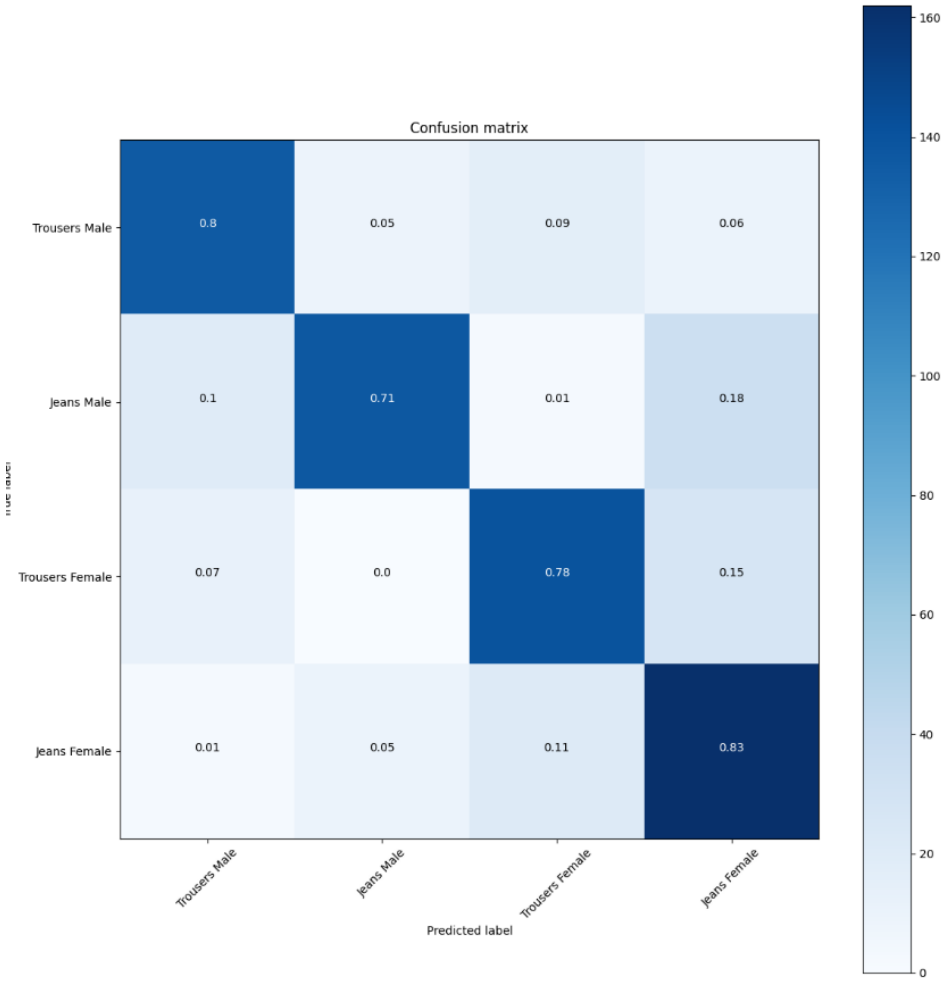


Figure 8: MobileNetV2 Confusion Matrix

5.3 ResNet50 Model Result Discussion

For the ResNet50 model, training accuracy gradually increased until peaking at the thirty-sixth epoch, with the loss decreasing to its minimum at the same step. Validation metrics remained relatively stable. The confusion matrix for ResNet50 as shown in figure 10 that the model correctly identifies 51% of Trousers Male and 51% of Jeans Male samples, with substantial misclassifications: 30% of Trousers Male as Trousers Female and 24% of Jeans Male as Jeans Female. For female clothing, the model performed better, correctly classifying 74% of Trousers Female and 62% of Jeans Female, though 28% of Jeans Female were misclassified as Trousers Female.

5.4 Model Comparison

Figure 11 compares training and validation accuracy across the Custom CNN (Run 9), MobileNetV2, and ResNet50 models. The Custom CNN achieves the highest accuracy early, reaching 94.1% training accuracy and 81.0% validation accuracy by epoch 4, with a smoothed validation accuracy of 80.2%. MobileNetV2 peaks at 88.1% training accuracy and 77.9% validation accuracy by epoch 10 (smoothed: 77.4%), while ResNet50 underperforms with 59.1% training and 59.4% validation accuracy at epoch 36 (smoothed: 58.2%).

Table 4 summarizes the performance metrics for all three models.

Across all three models, the Custom CNN demonstrates the strongest performance, achieving the highest training (94.11%), validation (81.03%), and test accuracy (77.19%). Its relatively higher validation and test

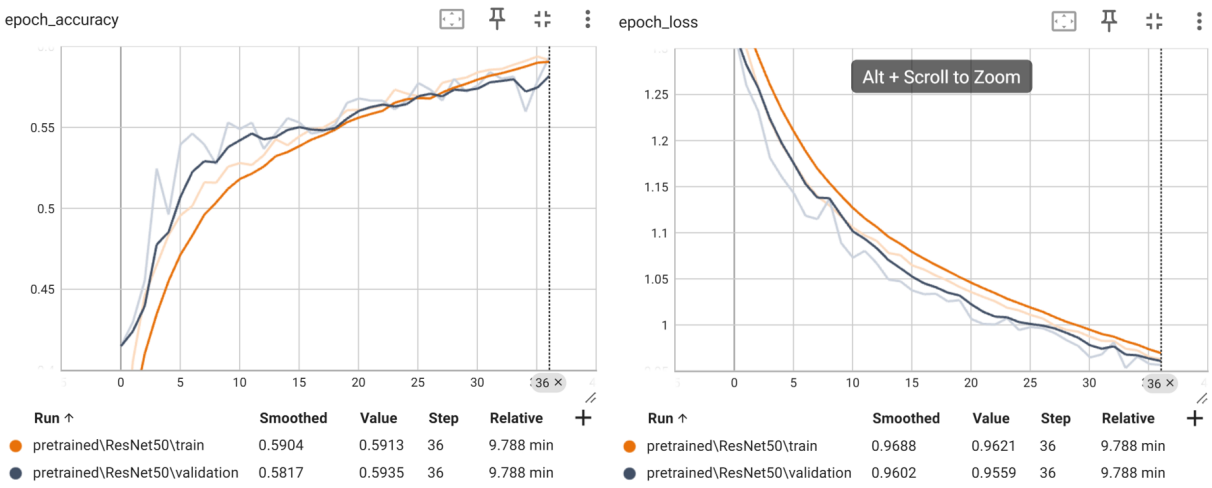


Figure 9: ResNet50 Accuracy & Loss

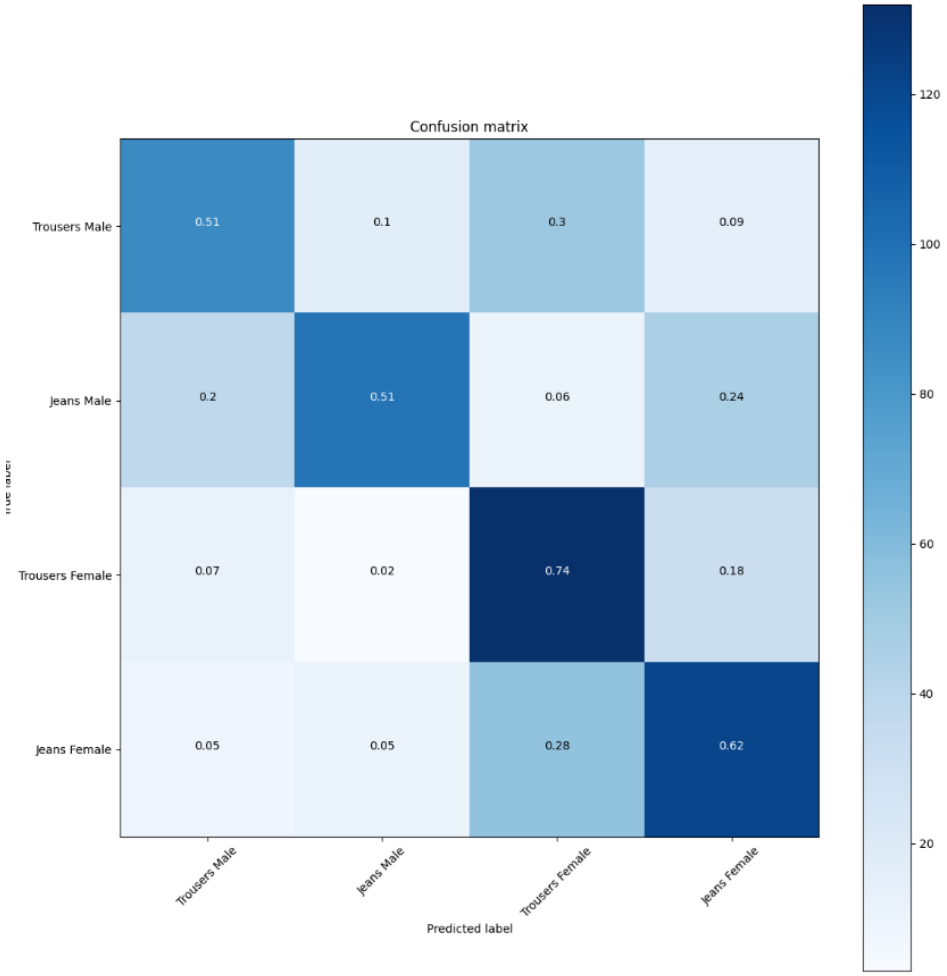


Figure 10: ResNet50 Confusion Matrix

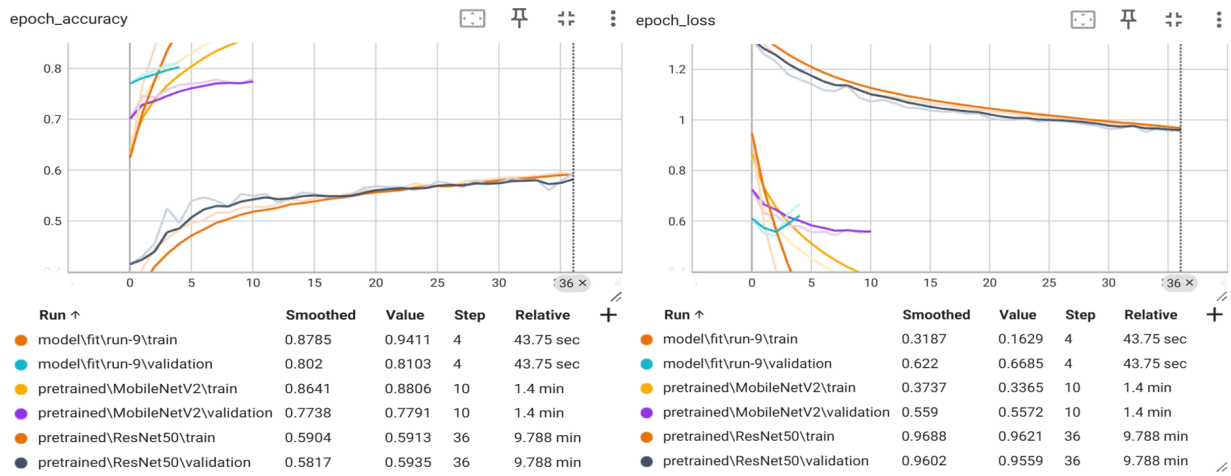


Figure 11: Model comparison plot

Table 4: Comparison of Custom CNN and Pretrained Models

Model	Train Acc	Train Loss	Val Acc	Val Loss	Test Acc	Test Loss
Custom CNN	0.9411	0.1629	0.8103	0.6685	0.7719	105.2653
MobileNetV2	0.8611	0.3775	0.7778	0.5448	0.2814	2.6194
ResNet50	0.5913	0.9621	0.5813	0.9559	0.2213	47.8568

losses are consistent with the use of early stopping, which was applied to prevent overfitting. MobileNetV2 shows moderate performance in training and validation, but exhibits poor generalization on the test set (28.14% test accuracy), likely due to the limited number of epochs. ResNet50 achieves the lowest accuracy across all metrics, reflecting the underfit caused by the limited duration of training despite its deep architecture. These results emphasize the efficiency and robustness of the Custom CNN in this classification task.

5.5 Conclusion

Despite extensive experimentation and optimization of the convolutional neural network architecture, the model continued to struggle with distinguishing between jeans and trousers. Upon closer inspection of the dataset (as shown in figure 4), it became evident that this challenge was not solely a model limitation but also a reflection of the inherent ambiguity in the visual characteristics of the two classes. Many of the training images displayed highly similar silhouettes, textures, and color patterns, making the distinction difficult even for human observers. As a result, the model's misclassifications can be understood as a natural outcome of trying to differentiate between categories with minimal discriminative visual cues.

This observation highlights an important aspect of image-based classification: model performance is fundamentally constrained by the clarity and separability of the underlying data. In real-world scenarios where classes share overlapping features, even state-of-the-art deep learning models may reach a performance ceiling. Therefore, rather than viewing this outcome as a failure, it should be interpreted as an honest reflection of the task's complexity. Future improvements may require higher-quality, more diverse images, additional metadata, or multimodal inputs to provide richer context beyond visual appearance alone.

References

[1] Leo Breiman. "Random forests". In: *Machine Learning* 45.1 (2001), pp. 5–32. DOI: 10.1023/A:1010933404324.

[2] Corinna Cortes and Vladimir Vapnik. "Support-vector networks". In: *Machine Learning* 20.3 (1995), pp. 273–297. DOI: 10.1007/BF00994018.

[3] Thomas Cover and Peter Hart. "Nearest neighbor pattern classification". In: *IEEE Transactions on Information Theory* 13.1 (1967), pp. 21–27. DOI: 10.1109/TIT.1967.1053964.

[4] Navneet Dalal and Bill Triggs. "Histograms of Oriented Gradients for Human Detection". In: *Computer Vision – ECCV 2005*. Vol. 3952. Lecture Notes in Computer Science. Springer, 2005, pp. 886–893. DOI: 10.1007/11499129_67.

[5] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016.

[6] Kaiming He et al. "Deep residual learning for image recognition". In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 2016, pp. 770–778. DOI: 10.1109/CVPR.2016.90.

[7] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. "ImageNet Classification with Deep Convolutional Neural Networks". In: *Advances in Neural Information Processing Systems*. Vol. 25. 2012, pp. 1097–1105.

[8] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. "Deep learning". In: *Nature* 521.7553 (2015), pp. 436–444. DOI: 10.1038/nature14539.

[9] David G. Lowe. "Distinctive image features from scale-invariant keypoints". In: *International Journal of Computer Vision* 60 (2004), pp. 91–110. DOI: 10.1023/B:VISI.0000029664.99615.94.

[10] Timo Ojala, Matti Pietikäinen, and Topi Mäenpää. "Multiresolution gray-scale and rotation invariant texture classification with local binary patterns". In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 24.7 (2002), pp. 971–987. DOI: 10.1109/TPAMI.2002.1017623.

[11] Mark Sandler et al. "MobileNetV2: Inverted Residuals and Linear Bottlenecks". In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 2018, pp. 4510–4520. DOI: 10.1109/CVPR.2018.00474.

[12] Josef Sivic and Andrew Zisserman. "Video Google: A text retrieval approach to object matching in videos". In: *Proceedings of the IEEE International Conference on Computer Vision*. 2003, pp. 1470–1477. DOI: 10.1109/ICCV.2003.1238693.

[13] Mingxing Tan and Quoc V. Le. "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks". In: *Proceedings of the 36th International Conference on Machine Learning*. Vol. 97. 2019, pp. 6105–6114. eprint: 1905.11946. URL: https://arxiv.org/abs/1905.11946.

[14] Jason Yosinski et al. "How transferable are features in deep neural networks?" In: *Advances in Neural Information Processing Systems*. Vol. 27. 2014, pp. 3320–3328.